

AI Solution Engineer

Post-Claim Pharmacy Financial Reconciliation

TIMEBOX 3-4 calendar days	TARGET EFFORT 12-16 focused hours	FORMAT Runnable prototype + concise design
DATA Synthetic only	DOMAIN US specialty pharmacy / 340B	DEBRIEF 20-30 minute walkthrough

What we want to evaluate

This exercise is designed to test how you approach an unfamiliar business domain, translate it into a sound technical architecture, and build a small AI-enabled solution that is **grounded in deterministic financial logic**. We care more about your decisions and reasoning than production polish.

The case

You are designing a prototype for a health-system specialty pharmacy operator. Once a claim has been filed, the operator needs to understand what money should have been received, what was actually received, what remains outstanding, and what requires action. A single claim may create pharmacy-benefit reimbursement, a 340B rebate, or a medical-benefit payment / denial pathway.

What a strong submission should demonstrate

- Ability to learn and explain the domain accurately, including where you are making assumptions.
- Clear separation between source-system data, deterministic reconciliation logic, and AI agent reasoning.
- A scalable architecture that can add more health systems, TPAs, payers and bank feeds without redesigning the solution.
- Useful agents that investigate, summarize or coordinate work, while preserving evidence and auditability.
- Good engineering judgement: scope control, tests, observability, failure handling and a credible path from prototype to production.

Confidentiality

The assignment is intentionally anonymized. Use synthetic data only. Do not use real patient information, client names, credentials, proprietary documents or production endpoints.

1. Business Problem and Assignment Scope

The prototype begins after a claim has been filed. Upstream activities such as patient intake, benefits investigation, prior authorization and claim creation are outside the assignment. Your job is to model the post-claim financial lifecycle and show how AI can assist operations without becoming the system of record for financial calculations.

Financial pathways to represent

Pathway	Illustrative source events	Financial question	Typical exception
Pharmacy benefit	PBM / PSAO adjudication, payment, reversal, adjustment, settlement	Was the expected reimbursement received and finally settled?	Underpayment, reversal, recoupment, missing settlement
340B recovery	TPA qualification, rebate request / status, manufacturer payment	Was the eligible rebate requested, approved and paid?	Rejected request, delayed status, unmatched rebate
Medical benefit	837 / clearinghouse status, payer response, 835 remittance, denial / appeal status	Was the claim paid, denied or still pending, and did cash arrive?	Denial, partial payment, missing 835, appeal outstanding
Cash / bank	ACH / deposit / bank reference and posting date	Can the financial event be tied to actual cash?	Remittance with no cash, unmatched deposit

Synthetic data requirement

Create a small synthetic dataset that is rich enough to test reconciliation and exception handling. You may use CSV, JSON, SQLite or a lightweight database. A script that generates the data is preferred so the results can be reproduced.

- At least 50 claim financial episodes across at least three source feeds. Four feeds are preferred: pharmacy/PBM, 340B/rebate, medical/remittance, and bank.
- Include at least six edge cases: fully reconciled, partial payment, underpayment, reversal/recoupment, unmatched cash or rebate, denial, duplicate event, or late-arriving status.
- Use tokenized or fictional member/patient identifiers. No real PHI.
- Preserve source identifiers so an investigator can trace a result back to the synthetic source record.

Suggested business object

Claim Financial Episode

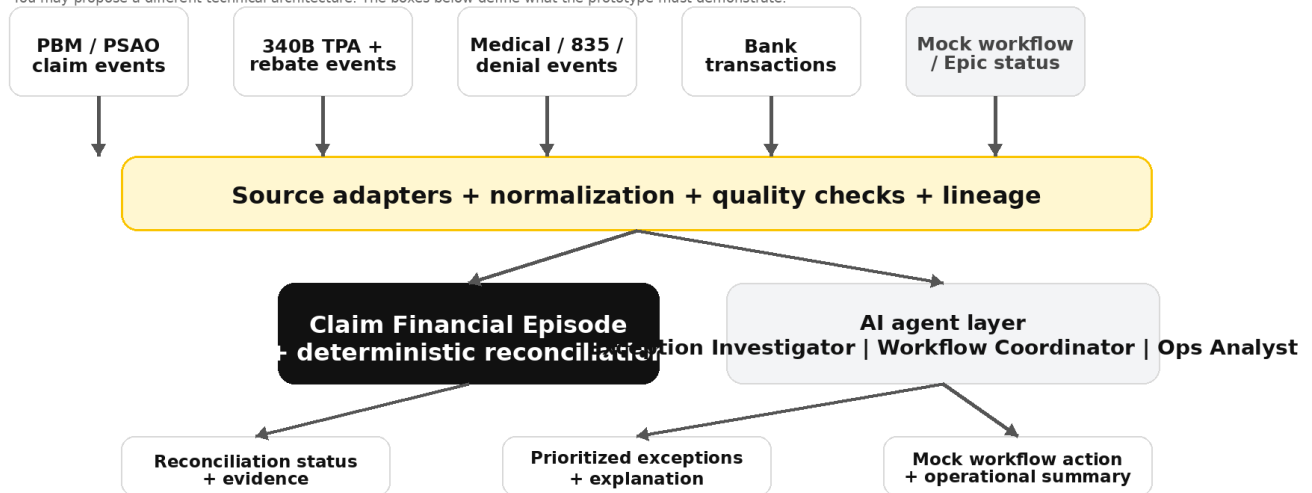
You may use a claim-centric object that relates the originating claim to reimbursements, 340B qualification and rebate events, medical remittance / denial events, bank cash, adjustments and exception states. You are free to propose a better model if you can explain why.

2. Architecture Challenge

Propose an architecture that could plausibly move from a prototype to a multi-tenant platform serving many health systems and source systems. The diagram below is only the functional boundary; we expect you to make and justify the technical choices.

Functional boundary for the assignment

You may propose a different technical architecture. The boxes below define what the prototype must demonstrate.



Your architecture should address

Source connectivity How APIs, SDKs, SFTP/files, remittances and bank feeds enter the platform. Show how you would handle schema versions, retries, duplicate delivery and late-arriving data.	Data foundation How raw source fidelity, normalized models, crosswalk keys, lineage and a claim-level semantic model are maintained.
Reconciliation services Where deterministic expected-vs-actual calculations, matching, cash allocation, reversals and close status live. These should not depend on an LLM.	Agent/tool boundary What tools/functions the agents can call, what they are not allowed to change, and how evidence is returned to the user.
Scale and tenancy How the design extends to additional TPAs, payers, health systems and reporting variants through configuration rather than one-off code.	Security and operations Authentication, tenant isolation, least-privilege access, audit logs, observability, data retention and safe handling of PHI in a real implementation.

Technology choice

Use any reasonable stack. We do not require a specific agent framework or model provider. If you use LangGraph, CrewAI, Semantic Kernel, OpenAI Agents SDK, a custom tool-calling loop, or another approach, explain why it fits this problem and what you would change for production.

3. Agent Challenge

Build at least two working agents or agent roles. They should operate on the synthetic data through explicit tools/functions. The goal is to see whether you can use AI where ambiguity and investigation are useful while keeping the financial truth deterministic.

Agent / role	Required behavior	Example question or action
Exception Investigator	Given an open item, retrieve the related claim, rebate/remittance, bank and workflow records; explain why it is open; cite the source evidence; identify missing information.	"Why is claim C-1048 still outstanding, and what evidence supports that conclusion?"
Workflow Coordinator	Recommend the next operational action and optionally create a mock work item. The agent must not close or financially post an item without deterministic confirmation.	"This medical claim is denied. Prepare the next-action summary and route a mock case to the Epic/RCM queue."
Ops / Portfolio Analyst	Answer aggregate questions over the portfolio and prioritize exceptions by value, age or category. This can be your second agent if you do not build Workflow Coordinator.	"Show the five highest-value unresolved items and summarize the dominant causes."

Minimum agent tools

Implement a small tool layer. The names below are examples, not a required interface:

- Retrieval tools for the claim episode, TPA/rebate status, remittance/denial and bank matches.
- `calculate_reconciliation(claim_id)` implemented as deterministic code/rules.
- Optional `create_mock_work_item(...)` for a workflow handoff.

Important constraint

The LLM may investigate, synthesize, classify, prioritize and recommend. Expected amounts, payment allocation, reconciliation status and ledger updates should come from deterministic logic and remain auditable.

Evaluation and grounding

- Create at least 10 test questions or scenarios with an expected answer or expected tool path.
- Return the supporting source evidence and say when the available data is insufficient.
- Capture a trace of tool calls and include at least one failure case with a production-hardening recommendation.

4. Deliverables and Suggested 3-4 Day Plan

Keep the submission focused. We are not looking for a production deployment or a polished front-end. A lightweight CLI, notebook, Streamlit page or simple web interface is sufficient if it makes the prototype easy to evaluate.

Required deliverables

Deliverable	Expected content
1. Short design note	Maximum 4-5 pages or 6-8 slides. Explain the domain, assumptions, target architecture, data model, agent/tool design, security considerations and what you would build next.
2. Runnable prototype	Repository with setup instructions and a working path from synthetic source data to reconciliation to at least two agent behaviors.
3. Synthetic data generator	Code or files that create the sample source feeds and intentionally inject exception scenarios.
4. Tests / evaluation	Automated tests for the reconciliation core plus a small agent evaluation set. Include the result summary.
5. README	Architecture diagram, prerequisites, how to run the demo, key assumptions, limitations and known trade-offs.

Suggested timebox

Day 1	Day 2	Day 3	Optional Day 4
Domain research, assumptions, source-event model, architecture and synthetic-data design.	Generate data, implement canonical model and deterministic reconciliation / exception logic.	Build two agents, connect tools, add evaluation, traces and a working demo path.	Polish docs/UI, add stretch capability or production-hardening notes. Do not overbuild.

Optional stretch goals

- A mocked direct-connector adapter with retry/idempotency behavior for a Beacon-like or TPA-like source.
- Human approval before workflow actions, with a simple audit/observability view.
- A short production roadmap covering connector onboarding, tenant configuration and monitoring.

Scope discipline matters

A smaller, correct solution is preferred over a broad but incomplete implementation.

5. Evaluation Rubric

We will assess the submission primarily on the quality of the reasoning, architecture and working behavior. The rubric below is also the structure we will use for the follow-up discussion.

Area	Weight	What we will look for
Domain understanding	15%	Correct framing of post-claim pathways, clear assumptions and distinction between remittance, rebate and cash.
Solution architecture	20%	Sound boundaries, connector strategy, canonical model, lineage, scale, security and operability.
Reconciliation correctness	20%	Deterministic calculations, matching, reversals/duplicates, clear statuses, tests and source evidence.
Agent design	20%	Useful roles, tool grounding, evidence-backed answers, safe action boundaries and failure handling.
Engineering quality	15%	Readable code, reproducible data, simple setup, modularity, tests and pragmatic choices.
Communication & prioritization	10%	Clear trade-offs, scope discipline and a concise explanation of what is built vs. next.

What is not required

- No real client data, live healthcare-system connectivity or production deployment.
- No production-grade UI, enterprise IAM buildout or large RAG corpus.
- No requirement to use multiple LLMs or a named multi-agent framework. Simpler is better if well reasoned.

Follow-up discussion

Be prepared for a 20-30 minute walkthrough. We will ask you to trace one claim end-to-end, defend the architecture, show how an agent obtained its evidence, and explain the production gaps.

Submission note

Include the code repository or archive, the design note and clear run instructions. If something cannot be run locally, provide screenshots or a short recording and explain why.